অধ্যায় ১৩

১৩

Object-Oriented Programming (OOP): Classes, Prototypes, Inheritance এবং Encapsulation

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Object-Oriented Programming-এর মূল ধারণা—তথ্য ও আচরণ একসঙ্গে—ব্যাখ্যা করতে।
- `class`, `constructor`, Field ও Method দিয়ে Object-এর ছাঁচ তৈরি করতে।
- Prototype Chain কীভাবে কাজ করে, এবং Class যে তারই ওপর একটি লেখার ধরন, তা বুঝতে।
- Private Field (`#`) দিয়ে Object-এর ভেতরের অবস্থা সুরক্ষিত রাখতে।
- Getter, Setter এবং Static সদস্য ব্যবহার করতে।
- `extends` ও `super` দিয়ে Inheritance এবং Method Override করতে।
- Polymorphism দিয়ে বিভিন্ন ধরনের Object একই ভাবে ব্যবহার করতে।
- Method আলাদা করে পাঠালে `this` হারানোর সমস্যা চিনে সমাধান করতে।
- কখন Inheritance আর কখন Composition, তা বিচার করতে।
- একটি সম্পূর্ণ **Booking Management Lab** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৭: Object, Method এবং `this`।
- অধ্যায় ১১: `class ... extends Error` দিয়ে নিজের Error (এখানে তার পূর্ণ ব্যাখ্যা)।
- অধ্যায় ১২: ES Module (`import` / `export`)।
- Node.js 22 বা নতুন সংস্করণ।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত আমরা তথ্য রেখেছি সাধারণ Object-এ, আর তার ওপর কাজ করেছি আলাদা Function দিয়ে: অধ্যায় ৭-এর শিক্ষার্থী, অধ্যায় ১২-এর কার্ট। এটি অনেক ক্ষেত্রেই সবচেয়ে ভালো পদ্ধতি। কিন্তু কিছু জিনিসের নিজস্ব নিয়ম থাকে, যা তথ্যের সঙ্গে অবিচ্ছেদ্য: একটি Bank Account-এর Balance কখনো ঋণাত্মক হবে না, একটি বাতিল Booking আবার বাতিল করা যাবে না। এমন ক্ষেত্রে তথ্য ও নিয়ম একসঙ্গে, একটি সুরক্ষিত একক হিসেবে রাখা সুবিধাজনক। এই ধারণার নাম **Object-Oriented Programming (OOP)**।

JavaScript-এর নিজের প্রায় সবকিছুই এভাবে তৈরি: প্রতিটি Array তার Method (`map`, `filter`) পায় একই উৎস থেকে, প্রতিটি Error-এর `name` ও `message` থাকে। Browser-এর DOM, Node.js-এর Stream—সবই Class। এই অধ্যায় শেষে আপনি শুধু Class লিখতেই পারবেন না, বুঝবেন ভেতরে কী ঘটে, এবং কখন Class ব্যবহার **না** করাই ভালো।

১৩.১ Factory Function থেকে Class

একটি Account তৈরির দুটি উপায় পাশাপাশি দেখুন:

factory-vs-class.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createAccount(owner, balance) {
  return {
    owner,
    balance,
    deposit(amount) {
      this.balance += amount;
    },
  };
}

class Account {
  constructor(owner, balance) {
    this.owner = owner;
    this.balance = balance;
  }

  deposit(amount) {
    this.balance += amount;
  }
}

const first = createAccount('Sara', 100);
const second = new Account('Rafi', 100);
first.deposit(50);
second.deposit(50);

console.log(first.balance, second.balance);
console.log(second instanceof Account, first instanceof Account);
```

```
const third = createAccount('Mina', 0);
const fourth = new Account('Nila', 0);
console.log(first.deposit === third.deposit, second.deposit === fourth.deposit);
```

Output

```
150 150
true false
false true
```

দুটিই কাজ করে, কিন্তু তিনটি পার্থক্য আছে:

- `new`: Class থেকে Object (Instance) তৈরি হয় `new` দিয়ে। `constructor` প্রতিটি নতুন Object-এর প্রাথমিক মান বসায়, আর ভেতরের `this` সেই নতুন Object।
- **পরিচয়**: `instanceof` বলতে পারে Object-টি কোন Class থেকে তৈরি। Factory-র Object শুধু একটি সাধারণ Object।
- **শেষ লাইনটি সবচেয়ে গুরুত্বপূর্ণ।** Factory প্রতিটি Object-এর জন্য `deposit`-এর একটি নতুন Copy তৈরি করেছে; Class-এর সব Instance একটিই `deposit` ভাগ করে নিয়েছে। কীভাবে, তা অনুচ্ছেদ ১৩.৩-এ।

১৩.২ Class-এর গঠন

class-basics.js · JavaScript চলবে: Browser Console ও Node.js

```
class Product {
  discount = 0;

  constructor(name, price) {
    this.name = name;
    this.price = price;
  }

  finalPrice() {
    return this.price * (1 - this.discount / 100);
  }

  label() {
    return `${this.name}: ${this.finalPrice()} টাকা`;
  }
}

const pen = new Product('Pen', 50);
pen.discount = 10;

console.log(pen.label());
```

```
console.log(Object.keys(pen).join(', '));
console.log(typeof Product);
```

Output

```
Pen: 45 টাকা
discount, name, price
function
```

অংশ	কী
`discount = 0;`	Public Field: প্রতিটি Instance-এ এই Property, এই প্রাথমিক মান নিয়ে
`constructor(...)`	`new`-এর সময় একবার চলে; Argument থেকে Property বসায়
`finalPrice() { }`	Method: সব Instance ভাগ করে নেয়; ভেতরে `this` মানে যে Instance-এর ওপর ডাকা হয়েছে

- Field-গুলো `constructor`-এর আগে বসে, তাই `Object.keys`-এ `discount` প্রথমে।
- একটি Method আরেকটিকে `this.finalPrice()` দিয়ে ডাকে।
- Class-এর নামের প্রথম অক্ষর বড় হাতের (`Product`), যাতে সাধারণ Function থেকে আলাদা করে চেনা যায়।
- `typeof Product` হলো `function`। Class আসলে একটি বিশেষ Function, তবে `new` ছাড়া ডাকলে `TypeError: Class constructor Product cannot be invoked without 'new'`।
- Class-এর ভেতরের সব Code নিজে থেকেই Strict Mode-এ চলে।

১৩.৩ Prototype Chain: Class-এর ভেতরে কী ঘটে

প্রতিটি JavaScript Object-এর একটি গোপন সংযোগ থাকে আরেকটি Object-এর সঙ্গে, যাকে বলে তার **Prototype**। কোনো Property Object-এ না পেলে JavaScript তার Prototype-এ খোঁজে, সেখানে না পেলে Prototype-এর Prototype-এ—এভাবে একটি শিকল ধরে, যতক্ষণ না `null`-এ পৌঁছায়। এটিই **Prototype Chain**।

prototype.js · JavaScript চলবে: Browser Console ও Node.js

```
class Product {
  constructor(name) {
    this.name = name;
  }

  label() {
    return `পণ্য: ${this.name}`;
  }
}

const pen = new Product('Pen');
```

```
console.log(Object.getPrototypeOf(pen) === Product.prototype);
console.log(Object.hasOwn(pen, 'name'), Object.hasOwn(pen, 'label'));
console.log('label' in pen);
console.log(Object.getPrototypeOf(Product.prototype) === Object.prototype);
console.log(typeof pen.toString);

Product.prototype.shout = function () {
  return this.label().toUpperCase();
};
console.log(pen.shout());
```

Output

```
true
true false
true
true
function
পণ্য: PEN
```

`new Product('Pen')` একটি Object তৈরি করেছে, যার নিজের Property শুধু `name`; তার Prototype হলো `Product.prototype`, যেখানে Class-এর সব Method থাকে। তাই `pen.label()` ডাকলে JavaScript প্রথমে `pen`-এ খোঁজে, না পেয়ে `Product.prototype`-এ পায়। শিকলটি:

ধাপ	Object	এখানে যা আছে
১	`pen`	`name` (নিজের)
২	`Product.prototype`	`constructor`, `label` (পরে `shout`)
৩	`Object.prototype`	`toString`, `hasOwnProperty` ইত্যাদি
৪	`null`	শিকলের শেষ

এ থেকে তিনটি জিনিস ব্যাখ্যা হয়: সব Instance একটিই Method ভাগ করে নেয় (অনুচ্ছেদ ১৩.১-এর শেষ লাইন); Prototype-এ পরে যোগ করা `shout` পুরোনো `pen`-ও পেয়েছে; আর অধ্যায় ১১-এর `catalog['toString']` সমস্যার উৎস এই শিকলের তৃতীয় ধাপ। `class` মূলত এই ব্যবস্থার ওপর একটি পরিষ্কার লেখার ধরন।

Class-এর আগের যুগ: Constructor Function

২০১৫-র আগে `class` ছিল না, এবং পুরোনো Library ও Project-এ আজও এই ধরন দেখবেন। ব্যবস্থাটি একই, শুধু হাতে লেখা:

constructor-function.js · JavaScript চলবে: Browser Console ও Node.js

```
function Product(name) {
  this.name = name;
}

Product.prototype.label = function () {
  return `পণ্য: ${this.name}`;
};

const pen = new Product('Pen');
console.log(pen.label(), pen instanceof Product);

const base = { greet() { return `হ্যালো, ${this.name}`; } };
const user = Object.create(base);
user.name = 'Sara';
console.log(user.greet(), Object.getPrototypeOf(user) === base);
```

Output

```
পণ্য: Pen true
হ্যালো, Sara true
```

একটি সাধারণ Function-কে `new` দিয়ে ডাকলে সেটি Constructor-এর কাজ করে, আর Method হাতে বসানো হয় `Product.prototype`-এ। `Object.create(base)` একটি নতুন Object তৈরি করে যার Prototype সরাসরি `base`, কোনো Class বা Constructor ছাড়াই। পুরোনো Code পড়ার সময় এগুলো চিনতে পারলেই যথেষ্ট; নতুন Code-এ `class` লিখুন, যা একই কাজ নিরাপদে করে (যেমন `new` ভুলে গেলে Error দেয়)।

> **সতর্কতা:** উদাহরণের মতো পরে Prototype-এ Method যোগ করা শুধু বোঝার জন্য। নিজের Code-এ Method Class-এর ভেতরেই লিখুন, আর `Array.prototype`-এর মতো Built-in Prototype কখনো বদলাবেন না: অন্য Code ও ভবিষ্যতের JavaScript সংস্করণের সঙ্গে সংঘর্ষ হয়।

১৩.৪ Encapsulation: Private Field

`account.balance = 1000000` লিখে যে-কেউ একটি সাধারণ Property বদলে দিতে পারে, নিয়ম না মেনেই। **Encapsulation** মানে Object-এর ভেতরের অবস্থা শুধু তার নিজের Method দিয়ে বদলানো যাবে। নামের আগে `#` দিলে Field বা Method **Private** হয়: Class-এর বাইরে থেকে তা পড়া বা লেখা যায় না।

private.js · JavaScript চলবে: Browser Console ও Node.js

```
class BankAccount {
  #balance = 0;

  deposit(amount) {
```

```
    this.#validate(amount);
    this.#balance += amount;
  }

  withdraw(amount) {
    this.#validate(amount);
    if (amount > this.#balance) {
      throw new RangeError('যথেষ্ট টাকা নেই');
    }
    this.#balance -= amount;
  }

  get balance() {
    return this.#balance;
  }

  #validate(amount) {
    if (!Number.isFinite(amount) || amount <= 0) {
      throw new RangeError('পরিমাণ ধনাত্মক হতে হবে');
    }
  }
}

const account = new BankAccount();
account.deposit(500);
account.withdraw(200);
console.log(account.balance, Object.keys(account).length);

try {
  account.withdraw(1000);
} catch (error) {
  console.log(error.message);
}
console.log(account.balance);
```

Output

```
300 0
যথেষ্ট টাকা নেই
300
```

- `#balance` শুধু Class-এর ভেতর থেকে পাওয়া যায়; `Object.keys`-এও দেখা যায় না (সংখ্যা `0`)। Balance বদলানোর একমাত্র পথ `deposit` ও `withdraw`, এবং দুটিই নিয়ম যাচাই করে।
- `#validate` একটি Private Method: Class-এর ভেতরের সহায়ক, বাইরের Code-এর জন্য নয়।
- `get balance()` বাইরের Code-কে মান পড়তে দেয়, লিখতে নয় (পরের অনুচ্ছেদ)।
- **ব্যর্থ `withdraw` অবস্থা বদলায়নি।** যাচাই আগে, বদল পরে: Object কখনো অর্ধেক-বদলানো অবস্থায় থাকে না।

বাইরে থেকে Private Field ছোঁয়ার চেষ্টা Program চলার আগেই ধরা পড়ে:

private-outside.js · JavaScript — চলবে: Node.js

```
class Safe {
  #code = 1234;
}

const safe = new Safe();
console.log(safe.#code);
```

Error Message

```
SyntaxError: Private field '#code' must be declared in an enclosing class
```

পুরোনো Code-এ Private বোঝাতে নামের আগে `_` দেওয়ার রীতি দেখবেন (`this._balance`)। সেটি শুধু একটি অনুরোধ; JavaScript তা মানতে বাধ্য করে না। `#` সত্যিকারের সুরক্ষা দেয়।

১৩.৫ Getter, Setter এবং Static

getter-static.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Temperature {
  static ABSOLUTE_ZERO = -273.15;

  #celsius = 0;

  constructor(celsius) {
    this.celsius = celsius;
  }

  get celsius() {
    return this.#celsius;
  }

  set celsius(value) {
    if (value < Temperature.ABSOLUTE_ZERO) {
      throw new RangeError(`${value} ডিগ্রি সম্ভব নয়`);
    }
    this.#celsius = value;
  }

  get fahrenheit() {
    return (this.#celsius * 9) / 5 + 32;
  }

  static fromFahrenheit(fahrenheit) {
```

```
    return new Temperature(((fahrenheit - 32) * 5) / 9);
  }
}

const today = new Temperature(30);
console.log(today.fahrenheit);
today.celsius = 35;
console.log(today.celsius, today.fahrenheit);

const boiling = Temperature.fromFahrenheit(212);
console.log(boiling.celsius);

try {
  today.celsius = -300;
} catch (error) {
  console.log(error.message);
}
console.log(typeof today.fromFahrenheit);
```

Output

```
86
35 95
100
-300 ডিগ্রি সম্ভব নয়
undefined
```

- **Getter (`get fahrenheit()`)** Property-র মতো পড়া যায় (Parenthesis ছাড়া), কিন্তু প্রতিবার হিসাব করে মান দেয়। তাই `celsius` বদলালে `fahrenheit` নিজে থেকেই ঠিক থাকে; দুটি আলাদা করে রাখলে একসময় অমিল হতো।
- **Setter (`set celsius(value)`)** সাধারণ Assignment-এর মতো লেখা যায়, কিন্তু ভেতরে যাচাই করে। `constructor`-ও `this.celsius = ...` লিখে Setter ব্যবহার করেছে, তাই তৈরির সময়ও একই নিয়ম।
- **Static** সদস্য Class-এর নিজের, Instance-এর নয়। `Temperature.ABSOLUTE_ZERO` একটি Class-স্তরের Constant; `Temperature.fromFahrenheit` একটি **Static Factory**, যা ভিন্ন ধরনের ইনপুট থেকে Instance তৈরি করে। Instance থেকে এগুলো পাওয়া যায় না (`undefined`)। `Array.isArray`, `Object.keys`, `Number.isInteger`—আপনার পরিচিত এগুলোও Static Method।

১৩.৬ Inheritance: extends এবং super

কিছু Class একই মূল ধারণার বিশেষ রূপ: রিকশা ও বাস দুটিই যানবাহন। **Inheritance** দিয়ে একটি Class আরেকটির সব বৈশিষ্ট্য নিয়ে নিজের পার্থক্যটুকু যোগ বা বদল করে।

inheritance.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Vehicle {
  constructor(name, wheels) {
    this.name = name;
    this.wheels = wheels;
  }

  describe() {
    return `${this.name}: ${this.wheels} চাকা`;
  }

  fare(km) {
    return km * 10;
  }
}

class Rickshaw extends Vehicle {
  constructor(name) {
    super(name, 3);
  }

  fare(km) {
    return Math.max(30, super.fare(km) * 2);
  }
}

class Bus extends Vehicle {
  constructor(name, seats) {
    super(name, 6);
    this.seats = seats;
  }

  describe() {
    return `${super.describe()}, ${this.seats} আসন`;
  }
}

const rickshaw = new Rickshaw('রিকশা');
const bus = new Bus('বাস', 40);

console.log(rickshaw.describe());
console.log(rickshaw.fare(1), rickshaw.fare(5));
console.log(bus.describe(), bus.fare(5));
console.log(bus instanceof Bus, bus instanceof Vehicle, bus instanceof Rickshaw);
```

Output

```
রিকশা: 3 চাকা
30 100
বাস: 6 চাকা, 40 আসন 50
true true false
```

- `class Rickshaw extends Vehicle` : `Rickshaw.prototype` -এর Prototype হলো `Vehicle.prototype` ; Prototype Chain-এ একটি ধাপ বাড়ল। তাই `rickshaw.describe()` নিজের Class-এ না পেয়ে `Vehicle` -এ পেয়েছে।
- `super(...)` Parent Class-এর `constructor` ডাকে। Child-এর `constructor` -এ `this` ব্যবহারের **আগে** এটি ডাকতে হয়।
- **Override**: Child একই নামের Method লিখলে সেটি Parent-এরটিকে ঢেকে দেয় (`Rickshaw` -এর `fare`)।
- `super.method()` Override করা Method-এর ভেতর থেকে Parent-এর সংস্করণ ডাকে, যাতে নিয়মটি নতুন করে লিখতে না হয়।
- `bus` একই সঙ্গে `Bus` এবং `Vehicle` , কিন্তু `Rickshaw` নয়।

`super()` -এর আগে `this` ব্যবহারের ফল:

this-before-super.js · JavaScript চলবে: Node.js

```
class Vehicle {
  constructor(name) {
    this.name = name;
  }
}

class Truck extends Vehicle {
  constructor(name, load) {
    this.load = load;
    super(name);
  }
}

new Truck('Truck', 5);
```

Error Message

```
ReferenceError: Must call super constructor in derived class
before accessing 'this'
```

১৩.৭ Polymorphism: একই প্রশ্ন, ভিন্ন উত্তর

Polymorphism মানে বিভিন্ন Class-এর Object একই Method-এর নামে সাড়া দেয়, প্রত্যেকে নিজের মতো করে। ব্যবহারকারী Code-কে জানতে হয় না কোনটি কোন Class।

polymorphism.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Payment {
  constructor(amount) {
    this.amount = amount;
  }

  fee() {
    return 0;
  }

  total() {
    return this.amount + this.fee();
  }

  label() {
    return `${this.constructor.name}: ${this.total()} টাকা`;
  }
}

class CardPayment extends Payment {
  fee() {
    return Math.round(this.amount * 0.02);
  }
}

class MobilePayment extends Payment {
  fee() {
    return Math.min(25, Math.round(this.amount * 0.015));
  }
}

const payments = [
  new Payment(500),
  new CardPayment(1000),
  new MobilePayment(3000),
];

for (const payment of payments) {
  console.log(payment.label());
}
console.log(`মোট Fee: ${payments.reduce((sum, payment) => sum + payment.fee(),
    0)}`);
```

Output

```
Payment: 500 টাকা
CardPayment: 1020 টাকা
MobilePayment: 3025 টাকা
মোট Fee: 45
```

`label` ও `total` শুধু `Payment`-এ লেখা, কিন্তু ভেতরের `this.fee()` প্রতিটি Object-এর নিজের Class-এর `fee` ডেকেছে। Loop জানে না কোনটি Card, কোনটি Mobile; জানার দরকারও নেই। নতুন একটি পরিশোধের ধরন যোগ করতে শুধু একটি নতুন Class লিখতে হবে, পুরোনো কোনো Code না বদলে। এই নীতির নাম **Open/Closed**: নতুন সংযোজনের জন্য খোলা, পুরোনো Code বদলের জন্য বন্ধ। `if (type === 'card') ... else if (type === 'mobile') ...` শৃঙ্খলের তুলনায় এটি অনেক নিরাপদ, কারণ নতুন ধরন যোগের সময় কোনো `if` ভুলে বাদ পড়ার সুযোগ নেই।

বাধ্যতামূলক Override: Abstract Method-এর ধরন

এখানে `Payment`-এর `fee()` একটি যুক্তিসঙ্গত Default (`0`) দেয়। কিন্তু কখনো Parent-এর পক্ষে কোনো যুক্তিসঙ্গত উত্তর থাকে না: একটি সাধারণ "আকৃতি"-র ক্ষেত্রফল কত? কিছু ভাষায় এমন Method-কে **Abstract** ঘোষণা করা যায়, যা প্রতিটি Child-কে লিখতেই হয়। JavaScript-এ সেই ব্যবস্থা নেই; প্রচলিত বিকল্প হলো Parent-এর Method-এ একটি স্পষ্ট ত্রুটি ছোড়া:

abstract.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Shape {
  area() {
    throw new Error(`${this.constructor.name} area() লেখেনি`);
  }
}

class Square extends Shape {
  constructor(side) {
    super();
    this.side = side;
  }

  area() {
    return this.side ** 2;
  }
}

class Triangle extends Shape {}

console.log(new Square(3).area());

try {
  new Triangle().area();
} catch (error) {
```

```
  console.log(error.message);
}
```

Output

```
9
Triangle area() লেখেনি
```

`Triangle`-এর লেখক `area()` লিখতে ভুলে গেছেন; ভুলটি নীরব `undefined` বা `NaN` না হয়ে প্রথম ব্যবহারেই একটি স্পষ্ট বার্তা দিয়েছে। বার্তায় `this.constructor.name` থাকায় বোঝা যায় কোন Class-এ ভুল।

১৩.৮ this হারানো: Method আলাদা করে পাঠালে

অধ্যায় ৭-এ দেখেছেন, `this` নির্ভর করে Method কীভাবে ডাকা হয়েছে তার ওপর। Class-এ এর একটি প্রচলিত ফাঁদ আছে:

lost-this.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Counter {
  count = 0;

  increment() {
    this.count += 1;
    return this.count;
  }

  incrementArrow = () => {
    this.count += 1;
    return this.count;
  };
}

const counter = new Counter();
const loose = counter.increment;

try {
  loose();
} catch (error) {
  console.log(error.name);
}

const bound = counter.increment.bind(counter);
console.log(bound());

const arrow = counter.incrementArrow;
console.log(arrow());
```

```
console.log([1, 2].map(() => counter.increment()).join(', '));
```

Output

```
TypeError
1
2
3, 4
```

`counter.increment`-কে একটি Variable-এ রেখে আলাদাভাবে ডাকলে সেটি আর `counter`-এর সঙ্গে যুক্ত নয়; Class-এর Strict Mode-এ `this` হয় `undefined`, এবং `this.count` পড়তে `TypeError`। একই ঘটনা ঘটে `setTimeout(counter.increment, 100)` বা `button.addEventListener('click', counter.increment)` লিখলে। তিনটি সমাধান:

১. **`bind(counter)`** একটি নতুন Function দেয়, যার `this` স্থায়ীভাবে `counter`।

২. **Arrow Function Field** (`incrementArrow = () => {}`): Arrow Function-এর নিজস্ব `this` নেই, তৈরির জায়গার `this`, অর্থাৎ Instance, ব্যবহার করে। তবে এটি প্রতিটি Instance-এ আলাদা Copy তৈরি করে, Prototype-এ থাকে না।

৩. **ডাকার জায়গায় Arrow Function**: `() => counter.increment()`। সবচেয়ে স্পষ্ট; সাধারণত এটিই ভালো।

১৩.৯ Composition বনাম Inheritance

Inheritance শক্তিশালী, কিন্তু তার একটি বড় দুর্বলতা আছে। ধরুন Order Service-কে গ্রাহককে বার্তা পাঠাতে হবে: Email-এ, SMS-এ, অথবা দুটোতেই। Inheritance দিয়ে লিখলে `EmailOrderService`, `SmsOrderService`, `EmailAndSmsOrderService`... প্রতিটি নতুন মাধ্যমে Class-এর সংখ্যা গুণ হারে বাড়ে। বিকল্প হলো **Composition**: Order Service-এর **একটি** বার্তা-প্রেরক **আছে**, যা বাইরে থেকে দেওয়া হয়।

composition.js · JavaScript চলবে: Browser Console ও Node.js

```
class EmailNotifier {
  send(to, text) {
    return `Email (${to}): ${text}`;
  }
}

class SmsNotifier {
  send(to, text) {
    return `SMS (${to}): ${text}`;
  }
}

class OrderService {
  #notifier;
```

```
  constructor(notifier) {
    this.#notifier = notifier;
  }

  place(order) {
    return this.#notifier.send(order.contact, `Order ${order.id} গৃহীত`);
  }
}

const byEmail = new OrderService(new EmailNotifier());
const bySms = new OrderService(new SmsNotifier());

console.log(byEmail.place({ id: 7, contact: 'sara@example.com' }));
console.log(bySms.place({ id: 8, contact: '01700-000000' }));
```

Output

```
Email (sara@example.com): Order 7 গৃহীত
SMS (01700-000000): Order 8 গৃহীত
```

`OrderService` জানে না বার্তা কীভাবে যায়; সে শুধু জানে তার প্রেরকের একটি `send` আছে। নতুন মাধ্যম (WhatsApp) যোগ করতে একটি নতুন Notifier Class, Order Service অপরিবর্তিত। Test-এ একটি নকল Notifier দেওয়া যায় (অধ্যায় ১২-এর Dependency Injection)। এমনকি `EmailNotifier` ও `SmsNotifier`-এর কোনো সাধারণ Parent Class নেই; শুধু একই আকারের Method থাকাই যথেষ্ট, এবং JavaScript-এ একেই বলে **Duck Typing**: "যদি হাঁসের মতো হাঁটে আর ডাকে, তবে সেটি হাঁস।"

প্রশ্ন	Inheritance (is-a)	Composition (has-a)
সম্পর্ক	রিকশা **একটি** যানবাহন	Order Service-এর **একটি** প্রেরক **আছে**
পরিবর্তনের নমনীয়তা	গঠন তৈরির সময় স্থির	চলার সময় বদলানো যায়
ঝুঁকি	গভীর শিকল; Parent বদলালে সব Child-এ প্রভাব	কম জড়ানো
কখন	স্পষ্ট "একটি বিশেষ ধরন" সম্পর্ক, অগভীর (এক-দুই স্তর)	আচরণ যোগ বা বদলাতে, প্রায় সব ক্ষেত্রে

প্রচলিত পরামর্শ: **Inheritance-এর আগে Composition বিবেচনা করুন।** Project-এ দুটিই আছে, প্রত্যেকটি তার উপযুক্ত জায়গায়।

১৩.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`new` ছাড়া Class ডাকা	`Class constructor ... cannot be invoked without 'new'`	`new Product()`
Child-এ `super()`-এর আগে `this`	`Must call super constructor ...`	প্রথমে `super(...)`
Child-এর `constructor`-এ `super()` নেই	একই `ReferenceError`	`super(...)` ডাকুন
Method-এর ভেতরে `this` ছাড়া Property	`ReferenceError` বা ভুল মান	`this.price`
Method আলাদা করে পাঠানো	`this` `undefined`, `TypeError`	`bind`, Arrow, বা `() => obj.method()`
Private Field বাইরে থেকে	`SyntaxError`	Getter বা Method
Getter-এর নামে Field-ও	অসীম পুনরাবৃত্তি (`Maximum call stack size exceeded`)	ভেতরের মান `#` নামে
Override করে Parent-এর নিয়ম হারানো	যাচাই বাদ পড়ে	`super.method()`
গভীর Inheritance শিকল	একটি বদলে অনেক Class ভাঙে	Composition
সব কিছুর জন্য Class	অকারণ জটিলতা	সাধারণ তথ্যে Object ও Function

১৩.১১ Debugging নির্দেশিকা

- **Console-এ Instance দেখুন।** Chrome ও Node.js Class-এর নাম দেখায় (`BankAccount {owner: 'Sara'}`); Chrome-এ `[[Prototype]]` খুলে Prototype Chain ধরে Method-গুলো দেখা যায়। Private Field Chrome-এর Console-এ দেখা যায়, কিন্তু Code থেকে পড়া যায় না।
- **"is not a function"** পেলে দেখুন Method-টি কোন Class-এ লেখা এবং Object-টি আসলে কোন Class-এর: `obj.constructor.name` এবং `obj instanceof X`।
- **`Cannot read properties of undefined (reading ...)` একটি Method-এর ভেতরে** প্রায় সব সময় হারানো `this`। Stack দেখুন Method-টি কে ডেকেছে: `setTimeout`, Event, নাকি `map`।
- **Override কাজ করছে না?** বানান মিলিয়ে দেখুন; ভুল বানানে নতুন একটি Method তৈরি হয়, Parent-এরটি চলতেই থাকে।
- **অবস্থা অপ্রত্যাশিত?** যে Method-গুলো Private Field বদলায়, শুধু সেগুলোতে Breakpoint দিন; Encapsulation-এর সুবিধা হলো খোঁজার জায়গা সীমিত।

১৩.১২ পেশাদার অভ্যাস

- **Class তখনই, যখন তথ্যের সঙ্গে নিয়ম ও অবস্থা আছে।** শুধু তথ্যের জন্য সাধারণ Object, শুধু হিসাবের জন্য Function।
- **Constructor-এ যাচাই:** কোনো Instance কখনো অবৈধ অবস্থায় তৈরি হবে না।
- **অবস্থা Private (`#`),** বাইরে শুধু প্রয়োজনীয় Getter ও Method।
- **হিসাব করা মান Getter-এ,** আলাদা Property-তে নয়।
- **Inheritance অগভীর;** আচরণ যোগে Composition।
- **Polymorphism দিয়ে `if (type === ...)` শৃঙ্খল এড়ান।**
- **Static Factory** ভিন্ন উৎস থেকে Instance তৈরিতে (`fromJSON`, `fromFahrenheit`)।
- **`toJSON`** দিয়ে ঠিক করুন Object কীভাবে সংরক্ষিত হবে; Private Field নিজে থেকে JSON-এ যায় না।

১৩.১৩ Project: Booking Management Lab

একটি ভ্রমণ সংস্থার Booking ব্যবস্থার মূল অংশ: Flight ও Hotel Booking, বাতিল করা, নিশ্চিত আয়ের হিসাব, এবং সংরক্ষণ ও পুনরুদ্ধার। Project-টি Node.js-এ চলে; এর মূল বিষয় Class-এর নকশা, তাই UI রাখা হয়নি। অধ্যায় ১২-এর মতো এটিও একটি Domain স্তর, যা পরে যেকোনো UI-এর সঙ্গে জোড়া যায়।

Project-এর চাহিদা

- প্রতিটি Booking-এর ID ও গ্রাহকের নাম খালি হবে না; দাম ধনাত্মক সংখ্যা।
- Flight-এর মোট = ভাড়া + Baggage Fee (ঋণাত্মক নয়)।
- Hotel-এর মোট = রাতপ্রতি দাম × রাতের সংখ্যা (১ বা বেশি পূর্ণসংখ্যা)।
- একই ID দুবার নয়; বাতিল Booking আবার বাতিল নয়; বাতিল Booking আয়ে ধরা হবে না।
- Booking-এর অবস্থা বাইরে থেকে সরাসরি বদলানো যাবে না।
- JSON-এ সংরক্ষণ এবং সঠিক Class-এ পুনরুদ্ধার, বাতিল অবস্থাসহ।
- প্রতিটি নতুন ও বাতিল Booking-এর খবর একটি Notifier-কে জানানো হবে (Composition)।

চাহিদা থেকে Class: নকশার ধাপ

Code লেখার আগে চাহিদার তালিকা থেকে নকশা বের করুন। একটি সহজ পদ্ধতি:

১. **বিশেষ্য খুঁজুন।** "Booking", "Flight Booking", "Hotel Booking", "Booking-এর তালিকা"। প্রতিটি সম্ভাব্য Class, কিন্তু সবগুলো নয়: গ্রাহকের নাম শুধু একটি লেখা, তার নিজের কোনো নিয়ম নেই, তাই সেটি Property।

২. **নিয়ম খুঁজুন।** "ID খালি হবে না", "দাম ধনাত্মক"—এগুলো Constructor-এর যাচাই, যাতে কোনো অবৈধ Booking তৈরিই না হয়। "বাতিল Booking আবার বাতিল নয়"—এটি একটি অবস্থার নিয়ম, তাই অবস্থাটি Private এবং বদলায় শুধু `cancel()` দিয়ে।

৩. **কী ধরন অনুযায়ী বদলায়, তা খুঁজুন।** মোট দামের সূত্র Flight ও Hotel-এ আলাদা, বাকি সব এক। এটিই Inheritance ও Polymorphism-এর সংকেত: সাধারণ অংশ Parent-এ, পার্থক্য (`total`, `type`) Child-এ।

৪. **কোনটি "is-a", কোনটি "has-a"।** Flight Booking **একটি** Booking: Inheritance। Manager-এর **একটি** Notifier **আছে**: Composition। Manager নিজে কোনো Booking নয়, তাই সে Booking থেকে Extend করে না; সে Booking-এর একটি তালিকা **রাখে**।

৫. **সীমানা খুঁজুন।** JSON-এ রূপান্তর একটি সীমানা (অধ্যায় ১১-এর Trust Boundary): বাইরে থেকে আসা তথ্য `restoreBooking`-এর অনুমোদিত তালিকা পেরিয়ে তবেই Class-এ ঢোকে।

এই পাঁচটি প্রশ্নের উত্তর থেকেই নিচের File-গুলোর গঠন এসেছে।

File-এর কাঠামো

Folder-এর কাঠামো

```
booking-lab/
├── src/
│   ├── booking.mjs       Booking, FlightBooking, HotelBooking, restoreBooking
│   └── manager.mjs       BookingManager
├── demo.mjs
├── README.md
└── tests/
    ├── booking.test.mjs
    └── manager.test.mjs
```

src/booking.mjs

দুই অংশ মিলে সম্পূর্ণ File।

booking.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
export class BookingError extends Error {
  constructor(message) {
    super(message);
    this.name = 'BookingError';
  }
}

function requireText(value, field) {
  if (typeof value !== 'string' || value.trim() === '') {
    throw new BookingError(`${field} খালি হতে পারবে না।`);
  }
  return value.trim();
}

function requirePositive(value, field) {
  if (!Number.isFinite(value) || value <= 0) {
    throw new BookingError(`${field} একটি ধনাত্মক সংখ্যা হতে হবে।`);
```

```
  }
  return value;
}

export class Booking {
  #status = 'confirmed';

  constructor({ id, customer, price }) {
    this.id = requireText(id, 'Booking ID');
    this.customer = requireText(customer, 'গ্রাহকের নাম');
    this.price = requirePositive(price, 'দাম');
  }

  get type() {
    return 'booking';
  }

  get status() {
    return this.#status;
  }

  get isActive() {
    return this.#status === 'confirmed';
  }

  get total() {
    return this.price;
  }

  cancel() {
    if (!this.isActive) {
      throw new BookingError(`${this.id} আগেই বাতিল হয়েছে।`);
    }
    this.#status = 'cancelled';
  }

  describe() {
    return `${this.id} (${this.type}) ${this.customer}: ${this.total} টাকা,
        ${this.#status}`;
  }

  toJSON() {
    const { id, customer, price } = this;
    return { type: this.type, id, customer, price, status: this.#status };
  }
}

// ...
```

booking.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

export class FlightBooking extends Booking {
  constructor({ baggageFee = 0, ...details }) {
    super(details);
    if (!Number.isFinite(baggageFee) || baggageFee < 0) {
      throw new BookingError('Baggage Fee ঋণাত্মক হতে পারবে না।');
    }
    this.baggageFee = baggageFee;
  }

  get type() {
    return 'flight';
  }

  get total() {
    return super.total + this.baggageFee;
  }

  toJSON() {
    return { ...super.toJSON(), baggageFee: this.baggageFee };
  }
}

export class HotelBooking extends Booking {
  constructor({ nights, ...details }) {
    super(details);
    if (!Number.isInteger(nights) || nights < 1) {
      throw new BookingError('রাতের সংখ্যা 1 বা বেশি পূর্ণসংখ্যা হতে হবে।');
    }
    this.nights = nights;
  }

  get type() {
    return 'hotel';
  }

  get total() {
    return this.price * this.nights;
  }

  toJSON() {
    return { ...super.toJSON(), nights: this.nights };
  }
}

const KINDS = { booking: Booking, flight: FlightBooking, hotel: HotelBooking };
```

```
// সংরক্ষিত JSON থেকে সঠিক Class-এর Booking, বাতিল অবস্থাসহ।
export function restoreBooking(data) {
  if (!Object.hasOwn(KINDS, data?.type)) {
    throw new BookingError(`অচেনা Booking-এর ধরন: ${data?.type}`);
  }
  const booking = new KINDS[data.type](data);
  if (data.status === 'cancelled') {
    booking.cancel();
  }
  return booking;
}
```

- `#status` **শুধু** `cancel()` **দিয়ে বদলায়,** এবং `cancel()` নিয়ম যাচাই করে। বাইরে থেকে `booking.status = 'confirmed'` লিখলে কিছুই হয় না, কারণ শুধু Getter আছে।
- `describe()` **একবার লেখা,** কিন্তু `this.type` ও `this.total` Getter প্রতিটি Child Class-এ Override করা। Flight-এর `describe()` তাই Flight-এর মোট দেখায়, Hotel-এরটি Hotel-এর (Polymorphism)।
- `super.total` : Flight-এর মোট Parent-এর মোটের ওপর ভিত্তি করে; ভাড়ার নিয়ম বদলালে একটি জায়গায় বদলাবে।
- `constructor({ baggageFee = 0, ...details })` : Destructuring ও Rest (অধ্যায় ৭) দিয়ে নিজের Field আলাদা করে বাকিটা `super` -কে। Parent-এর যাচাই (ID, নাম, দাম) Child-এর জন্যও প্রযোজ্য।
- `toJSON()` : `JSON.stringify` এটি নিজে থেকে ডাকে। Private `#status` নিজে থেকে JSON-এ যায় না, তাই এখানে স্পষ্টভাবে যোগ করা হয়েছে; Child Class `super.toJSON()` -এর সঙ্গে নিজের Field যোগ করে।
- `restoreBooking` : `type` দেখে সঠিক Class বেছে নেয়; `KINDS` একটি অনুমোদিত তালিকা, এবং `Object.hasOwn` অধ্যায় ১১-এর `toString` ফাঁদ এড়ায়।

src/manager.mjs

দুই অংশ মিলে সম্পূর্ণ File।

manager.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
import { Booking, BookingError, restoreBooking } from './booking.mjs';

export class BookingManager {
  #bookings = [];
  #notify;

  // Composition: Manager-এর একটি Notifier আছে, যা বাইরে থেকে দেওয়া হয়।
  constructor({ notify = () => {} } = {}) {
    this.#notify = notify;
  }

  add(booking) {
    if (!(booking instanceof Booking)) {
      throw new TypeError('শুধু Booking যোগ করা যায়');
```

```
    }
    if (this.find(booking.id)) {
      throw new BookingError(`${booking.id} আগে থেকেই আছে।`);
    }
    this.#bookings.push(booking);
    this.#notify(`নতুন: ${booking.describe()}`);
    return booking;
  }

  find(id) {
    return this.#bookings.find((booking) => booking.id === id) ?? null;
  }

  cancel(id) {
    const booking = this.find(id);
    if (!booking) {
      throw new BookingError(`${id} পাওয়া যায়নি।`);
    }
    booking.cancel();
    this.#notify(`বাতিল: ${id}`);
  }

  // ...
```

manager.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
  // ...

  get count() {
    return this.#bookings.length;
  }

  get confirmedRevenue() {
    return this.#bookings
      .filter((booking) => booking.isActive)
      .reduce((sum, booking) => sum + booking.total, 0);
  }

  revenueByType() {
    const revenue = {};
    for (const booking of this.#bookings.filter((item) => item.isActive)) {
      revenue[booking.type] = (revenue[booking.type] ?? 0) + booking.total;
    }
    return revenue;
  }

  list() {
    return [...this.#bookings];
  }
```

```
  toJSON() {
    return this.#bookings.map((booking) => booking.toJSON());
  }

  static fromJSON(items, options = {}) {
    const manager = new BookingManager();
    for (const item of items) {
      manager.add(restoreBooking(item));
    }
    manager.#notify = options.notify ?? (() => {});
    return manager;
  }
}
```

- `#bookings` **Private,** এবং `list()` একটি Copy ফেরত দেয়। বাইরের Code তালিকায় সরাসরি কিছু ঢোকাতে বা মুছতে পারে না; সব কিছু `add` ও `cancel`-এর নিয়ম মেনে।
- `confirmedRevenue` প্রতিটি Booking-এর `total` যোগ করে, না জেনে সেটি Flight না Hotel।
- `static fromJSON` একটি Static Factory। লক্ষ করুন, Static Method-এর ভেতরে `manager.#notify` লেখা গেছে: Private Field Class-এর **ভেতরের যেকোনো Code**-এর জন্য খোলা, এমনকি অন্য Instance-এর জন্যও। Notifier পুনরুদ্ধারের **পরে** বসানো হয়েছে, যাতে পুরোনো Booking ফিরিয়ে আনার সময় নতুন করে খবর না যায়।
- `instanceof Booking` যাচাই Polymorphism-এর সঙ্গে মেলে: যেকোনো Child Class-এর Booking গ্রহণযোগ্য।

demo.mjs

demo.mjs · JavaScript চলবে: Node.js

```
import { FlightBooking, HotelBooking } from './src/booking.mjs';
import { BookingManager } from './src/manager.mjs';

const messages = [];
const manager = new BookingManager({ notify: (text) => messages.push(text) });

manager.add(new FlightBooking({
  id: 'F-101', customer: 'Sara', price: 5200, baggageFee: 800,
}));
manager.add(new HotelBooking({
  id: 'H-201', customer: 'Rafi', price: 3500, nights: 3,
}));
manager.add(new FlightBooking({ id: 'F-102', customer: 'Mina', price: 4800 }));
manager.cancel('F-102');

for (const booking of manager.list()) {
  console.log(booking.describe());
}
console.log(`নিশ্চিত আয়: ${manager.confirmedRevenue} টাকা`);
```

```
console.log(manager.revenueByType());

try {
  manager.cancel('F-102');
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}

const saved = JSON.stringify(manager);
const restored = BookingManager.fromJSON(JSON.parse(saved));
console.log(`পুনরুদ্ধার: ${restored.count}টি, আয় ${restored.confirmedRevenue}`);
console.log(restored.find('H-201') instanceof HotelBooking, messages.length);
```

Output

```
F-101 (flight) Sara: 6000 টাকা, confirmed
H-201 (hotel) Rafi: 10500 টাকা, confirmed
F-102 (flight) Mina: 4800 টাকা, cancelled
নিশ্চিত আয়: 16500 টাকা
{ flight: 6000, hotel: 10500 }
BookingError: F-102 আগেই বাতিল হয়েছে।
পুনরুদ্ধার: 3টি, আয় 16500
true 4
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Notifier একটি সাধারণ Function** (`(text) => messages.push(text)`); Manager শুধু জানে এটিকে ডাকা যায়। বাস্তবে এখানে Email বা Log পাঠানো যেত, Manager না বদলে। চারটি বার্তা: তিনটি নতুন, একটি বাতিল।
- **`describe()` -এর তিনটি লাইন** তিনটি ভিন্ন হিসাব দেখায়: ভাড়া + Baggage, রাতপ্রতি × রাত, এবং বাতিল অবস্থা।
- **আয় ১৬,৫০০:** বাতিল F-102 বাদ। `revenueByType` প্রতিটি ধরনের আলাদা মোট দেয়।
- **দ্বিতীয়বার বাতিল** `Booking` -এর নিজের নিয়মে আটকেছে; Manager-কে এ নিয়ম জানতে হয়নি।
- **সংরক্ষণ ও পুনরুদ্ধার:** `JSON.stringify(manager)` Manager-এর `toJSON` , এবং তার ভেতরে প্রতিটি Booking-এর `toJSON` ডেকেছে। পুনরুদ্ধারে H-201 আবার একটি সত্যিকারের `HotelBooking` , এবং F-102 আবার বাতিল; আয় একই।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node demo.mjs
node --test
```

Call	যা হওয়ার কথা
`new Booking({ id: ' ', customer: 'A', price: 10 })`	`BookingError` : "Booking ID খালি হতে পারবে না।"
`new HotelBooking({ ..., nights: 1.5 })`	`BookingError` : রাতের সংখ্যার বার্তা
`new FlightBooking({ ..., baggageFee: -1 })`	`BookingError` : Baggage Fee-র বার্তা
একই ID দুবার `add`	`BookingError` : "... আগে থেকেই আছে।"
`manager.add({ id: 'X' })`	`TypeError` : সাধারণ Object গ্রহণযোগ্য নয়
`booking.status = 'confirmed'`	কিছুই বদলায় না (Module-এ `TypeError`)
`restoreBooking({ type: 'toString' })`	`BookingError` : অচেনা ধরন

Companion Source Code-এ ২০টি Automated Test আছে, যার মধ্যে Prototype Chain, Private Field-এর সুরক্ষা এবং JSON-এর মধ্য দিয়ে Class ও অবস্থা টিকে থাকার Test আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Hotel-এর মোট ভুল	`total` Override হয়নি বা বানান ভুল	Getter-এর নাম
`describe()` Child-এর মোট দেখায় না	`describe` -এ `this.total` -এর বদলে `this.price`	Getter ব্যবহার করুন
পুনরুদ্ধারের পর `instanceof` মিথ্যা	সাধারণ Object হিসেবে রাখা	`restoreBooking`
পুনরুদ্ধারের পর বাতিল Booking আবার নিশ্চিত	`toJSON` -এ `status` নেই	Private অবস্থা `toJSON` -এ
পুনরুদ্ধারের সময় অপ্রত্যাশিত বার্তা	Notifier আগে বসানো	পুনরুদ্ধারের পরে

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। টাকা পূর্ণসংখ্যা হিসেবে ধরা হয়েছে; তারিখ, আসন বা ঘরের প্রাপ্যতা নেই; তথ্য শুধু Memory-তে বা JSON লেখায়। Browser-এর UI অধ্যায় ১২-এর মতো একটি আলাদা Module হিসেবে এর ওপর তৈরি করা যায়।

১৩.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `Book` Class লিখুন: `title`, `author`, `pages` এবং `summary()` Method, যা `"বইয়ের নাম — লেখক (পৃষ্ঠা)"` ধরনের লেখা দেয়।

২. `Rectangle` Class লিখুন, যার `area` ও `perimeter` Getter।

৩. না চালিয়ে বলুন: `Object.hasOwn(pen, 'label')` কেন `false`, যদিও `pen.label()` কাজ করে?

৪. `const p = Product('Pen', 50);` কেন ব্যর্থ হয়? ঠিক করুন।

৫. `Wallet` Class লিখুন, যার Private `#balance`; `spend(amount)` বেশি খরচে `RangeError` দেয়।

মধ্যম স্তর

৬. `Shape` থেকে `Circle` ও `Square` তৈরি করুন, প্রতিটির নিজস্ব `area()`; একটি তালিকার মোট ক্ষেত্রফল দুই দশমিক পর্যন্ত দেখান।

৭. `Student` Class-এ `marks`-এর Setter লিখুন, যা ০ থেকে ১০০-এর বাইরে `RangeError` দেয়।

৮. নিচের `Clock`-এর `this` সমস্যা ঠিক করুন: `start() { setTimeout(this.tick, 10); }`।

৯. `Money.fromText('1,250 টাকা')` Static Factory লিখুন, যা লেখা থেকে সংখ্যা বের করে `Money` তৈরি করে।

চ্যালেঞ্জ স্তর

১০. Project-এ `BusBooking` যোগ করুন: `seats` (১ থেকে ৪০), মোট = ভাড়া × আসন। পুনরুদ্ধারও যেন কাজ করে।

১১. Inheritance ছাড়া ছাড়ের নিয়ম লিখুন: `NoDiscount`, `PercentDiscount`, `FlatDiscount`—প্রতিটির `apply(amount)`; একটি `PriceCalculator` যেকোনোটি নিয়ে কাজ করে।

১২. `Stack` Class লিখুন: Private Array, `push`, `pop`, `peek`, `size`; ফাঁকা Stack-এ `pop` ত্রুটি দেয়।

১৩.১৫ অনুশীলনীর সমাধান

সমাধান ১

book.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Book {
  constructor(title, author, pages) {
    this.title = title;
    this.author = author;
    this.pages = pages;
  }

  summary() {
    return `${this.title} — ${this.author} (${this.pages} পৃষ্ঠা)`;
  }
}

console.log(new Book('পথের পাঁচালী', 'বিভূতিভূষণ', 320).summary());
```

Output

```
পথের পাঁচালী — বিভূতিভূষণ (320 পৃষ্ঠা)
```

সমাধান ২

rectangle.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  get area() {
    return this.width * this.height;
  }

  get perimeter() {
    return 2 * (this.width + this.height);
  }
}

const room = new Rectangle(4, 3);
console.log(room.area, room.perimeter);
```

```
room.width = 5;
console.log(room.area);
```

Output

```
12 14
15
```

Getter প্রতিবার হিসাব করে, তাই প্রস্থ বদলানোর পর ক্ষেত্রফল নিজে থেকেই ঠিক।

সমাধান ৩ — `label` থাকে `Product.prototype`-এ, `pen`-এর নিজের Property হিসেবে নয়। `Object.hasOwn` শুধু নিজের Property দেখে; `pen.label()` কাজ করে Prototype Chain ধরে খোঁজার কারণে। `'label' in pen` হলে `true` হতো, কারণ `in` পুরো শিকল দেখে।

সমাধান ৪ — Class `new` ছাড়া ডাকা যায় না: `TypeError: Class constructor Product cannot be invoked without 'new'`। লিখুন `const p = new Product('Pen', 50);`।

সমাধান ৫

wallet.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Wallet {
  #balance;

  constructor(balance) {
    this.#balance = balance;
  }

  spend(amount) {
    if (amount > this.#balance) {
      throw new RangeError(`${amount} টাকা খরচ সম্ভব নয়; আছে ${this.#balance}`);
    }
    this.#balance -= amount;
    return this.#balance;
  }
}

const wallet = new Wallet(300);
console.log(wallet.spend(120));

try {
  wallet.spend(500);
} catch (error) {
```

```
  console.log(error.message);
}
```

Output

```
180
500 টাকা খরচ সম্ভব নয়; আছে 180
```

সমাধান ৬

shapes.js · JavaScript চলবে: Browser Console ও Node.js

```
class Shape {
  area() {
    return 0;
  }
}

class Circle extends Shape {
  constructor(radius) {
    super();
    this.radius = radius;
  }

  area() {
    return Math.PI * this.radius ** 2;
  }
}

class Square extends Shape {
  constructor(side) {
    super();
    this.side = side;
  }

  area() {
    return this.side ** 2;
  }
}

const shapes = [new Circle(1), new Square(2), new Circle(2)];
const total = shapes.reduce((sum, shape) => sum + shape.area(), 0);

console.log(total.toFixed(2));
```

Output

```
19.71
```

π + 8 + 8π = 9π + 8 ≈ ১৯.৭১। Loop জানে না কোনটি বৃত্ত, কোনটি বর্গ।

সমাধান ৭

student-marks.js · JavaScript চলবে: Browser Console ও Node.js

```
class Student {
  #marks = 0;

  constructor(name, marks) {
    this.name = name;
    this.marks = marks;
  }

  get marks() {
    return this.#marks;
  }

  set marks(value) {
    if (!Number.isFinite(value) || value < 0 || value > 100) {
      throw new RangeError(`নম্বর 0 থেকে 100-এর মধ্যে হতে হবে: ${value}`);
    }
    this.#marks = value;
  }
}

const student = new Student('Sara', 88);
console.log(student.marks);

try {
  student.marks = 120;
} catch (error) {
  console.log(error.message);
}
console.log(student.marks);
```

Output

```
88
নম্বর 0 থেকে 100-এর মধ্যে হতে হবে: 120
88
```

সমাধান ৮

clock.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Clock {
  ticks = 0;

  tick() {
    this.ticks += 1;
    console.log(`টিক ${this.ticks}`);
  }

  start() {
    setTimeout(() => this.tick(), 10);
  }
}

new Clock().start();
```

Output

```
টিক 1
```

`setTimeout(this.tick, 10)` শুধু Function-টি পাঠায়, Object নয়; পরে চলার সময় `this` হারিয়ে যায়। Arrow Function `this` ধরে রাখে, এবং ভেতরে `this.tick()` আবার Object-এর ওপর ডাকে।

সমাধান ৯

money.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Money {
  constructor(amount) {
    if (!Number.isInteger(amount) || amount < 0) {
      throw new RangeError(`অবৈধ অঙ্ক: ${amount}`);
    }
    this.amount = amount;
  }

  static fromText(text) {
    const digits = text.replace(/[^\d]/g, '');
    return new Money(Number(digits));
  }

  toString() {
    return `${this.amount.toLocaleString('en-US')} টাকা`;
  }
}
```

```
const price = Money.fromText('1,250 টাকা');
console.log(price.amount, `${price}`);
```

Output

```
1250 1,250 টাকা
```

Template Literal-এ Object বসালে JavaScript তার `toString()` ডাকে; Override করা `toString` তাই সুন্দর লেখা দেয়।

সমাধান ১০

bus-booking.mjs · JavaScript চলবে: Node.js

```
import { Booking, BookingError } from '../../project/booking-lab/src/booking.mjs';

export class BusBooking extends Booking {
  constructor({ seats, ...details }) {
    super(details);
    if (!Number.isInteger(seats) || seats < 1 || seats > 40) {
      throw new BookingError('আসন 1 থেকে 40-এর মধ্যে হতে হবে।');
    }
    this.seats = seats;
  }

  get type() {
    return 'bus';
  }

  get total() {
    return this.price * this.seats;
  }

  toJSON() {
    return { ...super.toJSON(), seats: this.seats };
  }
}

const trip = new BusBooking({
  id: 'B-1', customer: 'Nila', price: 650, seats: 3,
});
const saved = JSON.parse(JSON.stringify(trip));

console.log(trip.describe());
console.log(saved.type, saved.seats, saved.status);
```

Output

```
B-1 (bus) Nila: 1950 টাকা, confirmed
bus 3 confirmed
```

`describe`, `cancel`, যাচাই—সব Parent থেকে এসেছে; নতুন Class-এ শুধু পার্থক্যটুকু। পুনরুদ্ধারের জন্য `booking.mjs`-এর `KINDS` তালিকায় `bus: BusBooking` যোগ করতে হবে। `BookingManager` একটুও বদলাতে হয় না।

সমাধান ১১

discounts.js · JavaScript — চলবে: Browser Console ও Node.js

```
class NoDiscount {
  apply(amount) {
    return amount;
  }
}

class PercentDiscount {
  constructor(percent) {
    this.percent = percent;
  }

  apply(amount) {
    return Math.round(amount * (1 - this.percent / 100));
  }
}

class FlatDiscount {
  constructor(off) {
    this.off = off;
  }

  apply(amount) {
    return Math.max(0, amount - this.off);
  }
}

class PriceCalculator {
  #discount;

  constructor(discount = new NoDiscount()) {
    this.#discount = discount;
  }

  final(amount) {
    return this.#discount.apply(amount);
```

```
  }
}

const rules = [new NoDiscount(), new PercentDiscount(10), new FlatDiscount(200)];

for (const rule of rules) {
  const price = new PriceCalculator(rule).final(1500);
  console.log(`${rule.constructor.name}: ${price}`);
}
```

Output

```
NoDiscount: 1500
PercentDiscount: 1350
FlatDiscount: 1300
```

তিনটি ছাড়-Class-এর কোনো সাধারণ Parent নেই; `PriceCalculator` শুধু জানে প্রতিটির `apply` আছে (Duck Typing ও Composition)।

সমাধান ১২

stack.js · JavaScript চলবে: Browser Console ও Node.js

```
class Stack {
  #items = [];

  push(item) {
    this.#items.push(item);
    return this;
  }

  pop() {
    if (this.#items.length === 0) {
      throw new RangeError('Stack ফাঁকা');
    }
    return this.#items.pop();
  }

  peek() {
    return this.#items.at(-1);
  }

  get size() {
    return this.#items.length;
  }
}

const stack = new Stack().push('ক').push('খ').push('গ');
```

```
console.log(stack.pop(), stack.peek(), stack.size);
stack.pop();
stack.pop();

try {
  stack.pop();
} catch (error) {
  console.log(error.message);
}
```

Output

```
গ খ 2
Stack ফাঁকা
```

`push` `this` ফেরত দেয়, তাই পরপর ডাকা (Method Chaining) সম্ভব হয়েছে।

১৩.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `typeof` একটি Class-এর জন্য কী দেয়?

ক) `'class'`

খ) `'object'`

গ) `'function'`

ঘ) `'prototype'`

প্রশ্ন ২. একটি Class-এর Method কোথায় থাকে?

ক) Class-এর `prototype` Object-এ

খ) প্রতিটি Instance-এর নিজের Property হিসেবে

গ) Global Scope-এ

ঘ) `constructor`-এর ভেতরে

প্রশ্ন ৩. `#balance` Field বাইরে থেকে পড়ার চেষ্টা করলে কী হয়?

ক) `undefined`

খ) মানটি পাওয়া যায়

গ) `RangeError`

ঘ) `SyntaxError`, Program চলার আগেই

প্রশ্ন ৪. Child Class-এর `constructor`-এ `this` ব্যবহারের আগে কী ডাকতে হয়?

ক) `new`

খ) `super()`

গ) `this.constructor()`

ঘ) কিছুই না

প্রশ্ন ৫. `Temperature.fromFahrenheit()` কোন ধরনের Method?

ক) Private

খ) Getter

গ) Static

ঘ) Setter

প্রশ্ন ৬. বিভিন্ন Class-এর Object একই নামের Method-এ নিজের মতো সাড়া দেয়। এর নাম?

ক) Encapsulation

খ) Polymorphism

গ) Hoisting

ঘ) Composition

প্রশ্ন ৭. `setTimeout(counter.increment, 100)`-এ সমস্যা কী?

ক) Method-টি `counter`-এর সঙ্গে যুক্ত থাকে না; `this` হারায়

খ) `setTimeout` Method নিতে পারে না

গ) `increment` দুবার চলে

ঘ) কোনো সমস্যা নেই

প্রশ্ন ৮. "Order Service-এর একটি Notifier আছে"—এটি কোন সম্পর্ক?

ক) Inheritance

খ) Prototype

গ) Static

ঘ) Composition

প্রশ্ন ৯. `get total()` ব্যবহারের সুবিধা কোনটি?

ক) মান কখনো বদলায় না

খ) দ্রুত চলে

গ) Property-র মতো পড়া যায়, কিন্তু প্রতিবার সর্বশেষ তথ্য থেকে হিসাব হয়

ঘ) মান Private হয়ে যায়

> **প্রশ্ন ১০.** `JSON.stringify(booking)` Private Field কীভাবে সংরক্ষণ করবে?
>
> ক) নিজে থেকেই সব Private Field যায়
>
> খ) Class-এর `toJSON()` Method-এ স্পষ্টভাবে যোগ করতে হয়
>
> গ) Private Field কখনো সংরক্ষণ করা যায় না
>
> ঘ) `#` চিহ্নসহ সংরক্ষিত হয়

১৩.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–গ · ৬–খ · ৭–ক · ৮–ঘ · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	Class একটি বিশেষ Function।
২	ক	সব Instance Prototype Chain ধরে একই Method পায়।
৩	ঘ	Private নাম Class-এর বাইরে ব্যাকরণগতভাবেই অবৈধ।
৪	খ	Parent-এর `constructor` না চলা পর্যন্ত `this` তৈরি হয় না।
৫	গ	Class-এর ওপর ডাকা হয়, Instance-এর ওপর নয়।
৬	খ	একই প্রশ্ন (`fee()`), প্রতিটি Class-এর নিজের উত্তর।
৭	ক	Function আলাদা করে পাঠালে `this` আর Object নয়; `() => counter.increment()` লিখুন।
৮	ঘ	"আছে" (has-a) মানে Composition; "একটি ধরন" (is-a) মানে Inheritance।
৯	গ	হিসাব করা মান আলাদা করে রাখলে অমিলের ঝুঁকি থাকে।
১০	খ	`JSON.stringify` শুধু নিজের Public Property দেখে; `toJSON` তা নিয়ন্ত্রণ করে।

১৩.১৮ অধ্যায়ের সারসংক্ষেপ

- OOP তথ্য ও তার নিয়ম একসঙ্গে রাখে; Class একটি ছাঁচ, `new` দিয়ে Instance।
- Field ও `constructor` প্রতিটি Instance-এর অবস্থা বসায়; Method সবাই ভাগ করে নেয়।
- প্রতিটি Object-এর একটি Prototype; Property না পেলে JavaScript Prototype Chain ধরে খোঁজে; `class` এই ব্যবস্থার ওপর লেখার ধরন।
- Private (`#`) Field ও Method বাইরে থেকে অগম্য; অবস্থা বদলায় শুধু নিয়ম-মানা Method দিয়ে।
- Getter হিসাব করা মান দেয়, Setter যাচাই করে; Static সদস্য Class-এর নিজের, যেমন Static Factory।

- `extends` Prototype Chain-এ একটি ধাপ যোগ করে; `super()` আগে, `super.method()` দিয়ে Parent-এর নিয়ম পুনর্ব্যবহার।
- Polymorphism `if (type === ...)` শৃঙ্খলের বদলে; নতুন ধরন মানে নতুন Class, পুরোনো Code অপরিবর্তিত।
- Method আলাদা করে পাঠালে `this` হারায়; `bind`, Arrow Field, বা `() => obj.method()`।
- Inheritance (is-a) অগভীর রাখুন; আচরণ যোগে Composition (has-a) ও Duck Typing।
- `toJSON` ও Static `fromJSON` দিয়ে Class-এর Object সংরক্ষণ ও পুনরুদ্ধার।

পরের অধ্যায়ে Function-এর উন্নত ব্যবহার: Higher-Order Function, Composition, Currying, Memoization এবং Functional Programming-এর ধরন—Class-এর বিকল্প আরেকটি শক্তিশালী উপায়।

১৩.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Class	একই ধরনের Object তৈরির ছাঁচ।
Instance	Class থেকে `new` দিয়ে তৈরি Object।
Constructor	Instance তৈরির সময় চলা প্রাথমিক Code।
Prototype	যে Object থেকে আরেকটি Object Property উত্তরাধিকারসূত্রে পায়।
Prototype Chain	Property খোঁজার জন্য Prototype থেকে Prototype-এ শিকল।
Encapsulation	অবস্থা লুকিয়ে শুধু Method দিয়ে বদলানোর নীতি।
Private Field	`#` দিয়ে শুরু, Class-এর বাইরে অগম্য Field।
Getter / Setter	Property-র মতো পড়া ও লেখা, ভেতরে Code চলে।
Static	Class-এর নিজের সদস্য, Instance-এর নয়।
Inheritance	`extends` দিয়ে আরেকটি Class-এর বৈশিষ্ট্য নেওয়া (is-a)।
Polymorphism	একই Method-এ বিভিন্ন Class-এর ভিন্ন আচরণ।
Composition	অন্য Object-কে অংশ হিসেবে রেখে আচরণ গঠন (has-a)।

১৩.২০ আরও পড়ুন

- **MDN — Classes:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes` — Class-এর সম্পূর্ণ রেফারেন্স।

- **MDN — Private properties:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_properties` — `#` Field ও Method।
- **MDN — Inheritance and the prototype chain:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain` — Prototype Chain-এর গভীর ব্যাখ্যা।
- **MDN — extends:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/extends` — Inheritance ও `super`।
- **MDN — Function.prototype.bind():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/bind` — `this` স্থির করা।